

Cheatsheet

One row per function: the call, with its arguments, and what it does. Grouped by topic, with the imports a topic needs at the top of its card and the session that introduced each row at its edge.

Running code

A notebook shows the value of the last line of a cell. Anything else you want to see, you have to print.

<code>print(value, other)</code>	Show one or more values. Separate several with commas.	Session 1
<code># comment</code>	Everything after <code>#</code> on a line is a note for humans, not code.	Session 1
<code>value</code>	The last line of a cell is displayed automatically, with no print.	Session 1

Numbers

<code>+</code> <code>.</code> <code>-</code> <code>*</code> <code>/</code>	The usual arithmetic. Division always gives a float.	Session 1
<code>x ** n</code>	To the power of. Compound growth, and square roots as <code>** 0.5</code> .	Session 1
<code>round(number, ndigits)</code>	Round to a number of decimals.	Session 1
<code>abs(number)</code>	Distance from zero, sign discarded.	Session 2
<code>x // n</code> <code>x % n</code>	Whole division and the remainder: <code>7 // 5</code> is 1 and <code>7 % 5</code> is 2. They turn a position counted along a grid's rows into its row and its column.	Session 11

Text

<code>"text"</code>	A string. Single or double quotes, as long as they match.	Session 1
<code>"AA" + "PL"</code>	Glue strings together. Both sides must be strings.	Session 1
<code>f"{value}"</code>	An f-string: put a value inside text.	Session 1

<code>f"{x:.2%}"</code>	As a percentage with 2 decimals. The recipe for returns.	Session 1
<code>f"{x:.2f}"</code>	As a plain number with 2 decimals. The recipe for prices.	Session 1
<code>f"{name:<6}"</code> · <code>f"{x:>7.1%}"</code>	A field width: pad left-aligned text, or right-align a number, so a column of output lines up.	Session 2
<code>text.upper()</code> · <code>text.lower()</code>	A copy of the string in one case. Tickers arrive in every mixture of both.	Session 1
<code>text.strip()</code>	A copy with the spaces at each end removed. The first thing you do to text from a file.	Session 2
<code>text.count(part)</code>	How many times a piece of text appears inside a string.	Session 1
<code>text.startswith(part)</code> · <code>text.endswith(part)</code>	True when the text begins, or ends, with that piece. [c for c in columns if c.startswith("vol_")] keeps the columns whose names begin with vol_.	Session 3

True and false

<code>==</code> · <code>!=</code>	Equal, and not equal. Two equals signs, because one assigns.	Session 1
<code><</code> · <code>></code> · <code><=</code> · <code>>=</code>	The usual comparisons. The answer is True or False.	Session 1
<code>and</code> · <code>or</code> · <code>not</code>	Combine conditions. In pandas the same job is done by <code>&</code> , <code> </code> and <code>~</code> .	Session 2

Types

<code>type(value)</code>	What kind of thing this is.	Session 1
<code>int(x)</code> · <code>float(x)</code> · <code>str(x)</code>	Convert between whole number, decimal and text.	Session 1
<code>None</code>	The absence of a value. What a function returns when it has no return.	Session 1

Built-in functions

Always available, no import needed.

<code>len(items)</code>	How many items.	Session 1
-------------------------	-----------------	-----------

<code>sum(numbers)</code>	Add them all up.	Session 1
<code>min(values)</code> · <code>max(values)</code>	Smallest and largest.	Session 1
<code>sorted(values)</code>	A new list, in order. The original is untouched.	Session 2
<code>range(start, stop)</code>	Whole numbers from start up to but NOT including stop.	Session 2
<code>enumerate(items)</code>	The position and the value together, so you need no counter of your own.	Session 2
<code>zip(a, b)</code>	Walk two lists in step, with no indexing. Stops at the shorter one.	Session 2
<code>set(items)</code>	Each distinct item once, in no fixed order. <code>len(set(items))</code> counts the distinct items.	Session 9
<code>help(thing)</code>	Print the documentation. Works offline.	Session 2
<code>dir(thing)</code>	List everything an object can do. Useful when you half-remember a name.	Session 3

Lists

An ordered box of values. Counting starts at 0.

<code>[a, b, c]</code>	Make a list.	Session 1
<code>items[i]</code>	The item at that position. [0] is first.	Session 1
<code>items[-1]</code>	Counting from the end. [-1] is last.	Session 1
<code>items[a:b]</code>	A slice: from a up to but NOT including b.	Session 1
<code>items[:n]</code> · <code>items[n:]</code>	Everything before n, everything from n.	Session 1
<code>items.append(x)</code>	Add to the end. Changes the list itself.	Session 1
<code>items.index(x)</code>	The position of the first x.	Session 1
<code>[]</code>	An empty list, ready for a loop to fill.	Session 1
<code>[x for x in items if condition]</code>	A list built in one line: every item for which the condition holds. <code>[c for c in frame.columns if c != "price"]</code> is every column but the target. Without the if it keeps them all, and <code>x * 2</code> in place of the first x changes each one.	Session 3

Loops

Doing something once per item.

<code>for item in items:</code>	Run the indented body once for every item.	Session 2
<code>for i in range(n):</code>	Loop over positions rather than values.	Session 2
<code>for i in range(1, len(items)):</code>	Positions 1 onwards: how you reach item i and i-1 together.	Session 2
<code>total += x</code>	Add to a variable and store it back. Same as <code>total = total + x</code> .	Session 2
<code>while condition:</code>	Repeat for as long as the condition holds. Something inside must change it.	Session 2

Making choices

<code>if condition:</code>	Run the block only when the condition is True.	Session 2
<code>elif condition: · else:</code>	Further cases, checked in order, and a catch-all.	Session 2

Your own functions

<code>def name(parameters):</code>	Create a function. The indented body is what it does.	Session 2
<code>return value</code>	Hand a value back and stop there. Without it you get None.	Session 2
<code>def name(x, rate=0.05):</code>	A default: used when the caller does not pass it.	Session 2
<code>"""What it does."""</code>	A docstring, the first line of the body. Editors show it on hover, and <code>help()</code> prints it.	Session 2

Dictionaries

Values stored by name instead of by position.

<code>{"AAPL": 0.36, "KO": 0.12}</code>	Make one. Keys are usually strings.	Session 2
<code>data[key]</code>	Read the value under that key.	Session 2
<code>data[key] = value</code>	Add a new key, or overwrite an existing one.	Session 2

<code>for key in data:</code>	Loop over the keys. Read each value with <code>data[key]</code> .	Session 2
<code>data.keys()</code> · <code>data.values()</code>	The names on their own, or the numbers on their own. <code>max</code> , <code>min</code> , <code>sum</code> and <code>for</code> all accept them.	Session 2
<code>for key, value in data.items():</code>	The key and the value together, so you need no lookup.	Session 2
<code>{}</code>	An empty dictionary, ready for a loop to fill. The counterpart of <code>[]</code> .	Session 2
<code>min(data, key=data.get)</code>	The KEY with the smallest value, not the value itself. <code>max</code> works the same way.	Session 5
<code>sorted(data, key=data.get)</code>	All the keys, ordered by their values. Largest first with <code>reverse=True</code> .	Session 5
<code>{key: round(value, 3) for key, value in data.items()}</code>	A dictionary built in one line, one entry per item: here the same keys with every value rounded.	Session 3
<code>name(**settings)</code>	Call a function, or create a model, with the keys of a dictionary as the argument names: <code>XGBRegressor(**{"max_depth": 3})</code> is <code>XGBRegressor(max_depth=3)</code> . One dictionary of settings then reaches every fit that needs it.	Session 12

Packages

<code>import numpy as np</code>	Bring a package in under a short name. <code>np</code> , <code>pd</code> and <code>plt</code> are conventions everybody uses.	Session 3
<code>%pip install pandas</code>	Install a package. Once per machine, in a notebook cell.	Session 3

NumPy arrays

A list that does arithmetic on every element at once. Indexing and slicing work exactly as they do for lists.

```
import numpy as np
```

On this card: `values` is a one-dimensional array, `mask` an array of `True` and `False`.

<code>np.array(list)</code>	Turn a list of numbers into an array.	Session 3
<code>values * 2</code>	Arithmetic applies to every element. No loop.	Session 3

<code>a - b</code>	Two arrays, element by element.	Session 3
<code>(v[1:] - v[:-1]) / v[:-1]</code>	Every daily return in one line: each day against the day before.	Session 3
<code>values.mean()</code>	The average.	Session 3
<code>values.std()</code>	The standard deviation. This is what volatility is.	Session 3
<code>values.min()</code> · <code>values.max()</code> · <code>values.sum()</code>	The usual summaries.	Session 3
<code>np.sqrt(x)</code>	Square root. <code>np.sqrt(252)</code> annualises a daily volatility.	Session 3
<code>values > 0</code>	A question asked of every element: True or False for each.	Session 3
<code>(values > 0).sum()</code>	Count the Trues, because True counts as 1.	Session 3
<code>(values > 0).mean()</code>	The share that are True.	Session 3
<code>values[values > 0]</code>	Keep only the elements where the condition holds.	Session 3
<code>values.shape</code>	How big it is. For a 1-D array it prints as (n,).	Session 3
<code>values.argmax()</code> · <code>values.argmin()</code>	The POSITION of the largest or smallest element. Feed it to an index to get the label.	Session 5
<code>(a == b).all()</code>	True only when every element of a comparison is True: the vectorised way to check that two arrays or columns agree everywhere.	Session 8
<code>np.allclose(a, b)</code> · <code>np.isclose(a, b)</code>	Equal up to rounding error: one True or False for the whole array, or one per element. Two computations of the same number rarely agree to the last decimal, so <code>==</code> is the wrong test for floats.	Session 3
<code>float(values.mean())</code>	A plain Python number from a NumPy one. From NumPy 2 on, a NumPy number stored in a list or a dictionary prints as <code>np.float64(0.2)</code> ; after <code>float()</code> it prints as 0.2.	Session 5
<code>np.diff(values)</code>	The change from each element to the next, so n values give n - 1 changes. <code>(np.diff(values) < 0).all()</code> is True when the values fall at every step.	Session 12
<code>np.where(mask, a, b)</code>	An array that takes a where the mask is True and b elsewhere. Keeps one group's values and blanks the rest before <code>argmin</code> or <code>argmax</code> .	Session 8

<code>np.zeros(n, dtype=int)</code>	An array of n zeros, as integers; without dtype, as floats. A prediction of 0 on every row is <code>np.zeros(len(test), dtype=int)</code> .	Session 8
<code>np.arange(start, stop, step)</code>	Values from start up to but not including stop, a fixed step apart. <code>np.linspace</code> fixes the count instead of the step.	Session 8
<code>np.logspace(0, 4, 5)</code>	Values spaced by a constant factor: 1, 10, 100, 1000, 10000. The right shape for a grid of alphas.	Session 6
<code>np.unique(values)</code>	The distinct values, sorted. A cut on a column can only fall between two neighbouring ones.	Session 11
<code>np.quantile(values, np.linspace(0.02, 0.98, 60))</code>	Sixty cut points spread evenly through the data, from the 2nd to the 98th percentile. <code>s.quantile</code> does the same for a Series.	Session 11
<code>np.asarray(values, dtype=float)</code>	An array of floats from a list, an array or a Series, so one function accepts all three.	Session 11
<code>np.mean(arrays, axis=0)</code>	Average several arrays of the same length, position by position: many trees' forecasts made into one.	Session 11
<code>values.cumsum(axis=0)</code>	Running totals down the rows of a 2-D array. Divide row n by n + 1 for the running average of the first rows: how a forest's error moves as trees are added.	Session 11
<code>np.random.default_rng(0)</code>	A random number generator with a fixed seed, so the same code gives the same numbers every time.	Session 4
<code>rng.normal(0, 1, size=n)</code>	n draws from a normal distribution with mean 0 and standard deviation 1, from <code>rng = np.random.default_rng(0)</code> . As a column of a table: pure noise.	Session 4
<code>rng.integers(0, 2, size=n)</code> · <code>rng.permutation(values)</code>	n whole numbers from 0 up to but not including 2, a coin for each row; and the values given, in a random order. Shuffling one column of the test rows is how permutation importance breaks it.	Session 12

NumPy matrices

A matrix is a 2-D array: rows first, then columns.

```
import numpy as np
```

<code>np.array([[1, 2], [3, 4]])</code>	A matrix, written as a list of rows.	Session 3
<code>X.shape</code>	Rows and columns.	Session 3
<code>X[i, j] · X[i, :] · X[:, j]</code>	One element, a whole row, a whole column. The colon means all.	Session 3
<code>X.T</code>	The transpose: rows become columns.	Session 3
<code>X.sum(axis=1)</code>	Sum along each row, one total per row; axis=0 sums down each column instead. The softmax divides every row of scores by its own total, so each row of probabilities adds to one.	Session 9
<code>A @ B</code>	Matrix multiplication. For two vectors this is the dot product.	Session 3
<code>np.ones(n)</code>	A vector of ones. The intercept column of a regression.	Session 3
<code>np.column_stack([a, b])</code>	Glue 1-D arrays together as the columns of a matrix.	Session 3
<code>np.linalg.inv(A)</code>	The inverse: the matrix that undoes A. Fails if a column is redundant.	Session 3
<code>np.linalg.solve(A, b)</code>	Solve $Ax = b$. Safer and faster than inverting. OLS in one line: <code>solve(X.T @ X, X.T @ y)</code> .	Session 3

pandas: one column

A Series is values with a label on each one: a list and a dictionary at once.

```
import pandas as pd
```

On this card: s is a Series.

<code>pd.Series(a_dict)</code>	Build one from a dictionary. The keys become the labels.	Session 3
<code>s["AAPL"]</code>	Look a value up by its label.	Session 3
<code>s * 2</code>	Maths applies to every value, and the labels come along.	Session 3
<code>s.mean() · s.std() · s.min() · s.max()</code>	A Series summarises itself, like an array.	Session 3
<code>s.median()</code>	The middle value. Compare it with the mean: a gap between them means a skewed tail.	Session 4

<code>s.sort_values(ascending=False)</code>	In order. Leave ascending out for smallest first.	Session 3
<code>s.value_counts(normalize=True)</code>	The share of each distinct value, largest first; leave normalize out for the counts. The largest share is what the majority rule scores.	Session 9
<code>s.dropna()</code>	Throw away the missing values.	Session 3
<code>s.unique()</code>	The distinct values, in the order they first appear. <code>sorted()</code> puts them in order.	Session 3
<code>s.nunique()</code>	How many distinct values there are. On the index of a bootstrap sample: how many different days it drew.	Session 11
<code>s.sort_index()</code>	Order by the labels instead of the values. After <code>value_counts</code> , the counts in order of the value counted.	Session 3
<code>s.isin(values)</code>	True where the value appears in values. <code>frame.index.isin(sample.index)</code> marks the days a sample drew, and <code>~</code> the days it left out.	Session 11
<code>s.value_counts().reindex(index, fill_value=0)</code>	A count for every label in index, with 0 for labels that never appeared: how many times a bootstrap sample drew each day.	Session 11
<code>s.abs()</code>	The size of every value, sign dropped. <code>abs()</code> on its own does the same for one number.	Session 4
<code>s.idxmin()</code> · <code>s.idxmax()</code>	The LABEL of the smallest or largest value, where <code>argmin</code> gives its position: on a Series of errors indexed by alpha, the alpha with the lowest error.	Session 4
<code>s.to_dict()</code>	The Series as a plain dictionary, label to value. <code>s.value_counts().to_dict()</code> keeps the counts in a form any report can store.	Session 3

pandas: a table

A DataFrame is several Series side by side, sharing one index.

```
import numpy as np
import pandas as pd
```

On this card: frame is a DataFrame, mask a column of True and False.

<code>pd.read_csv(path, parse_dates=["date"])</code>	Read a CSV. <code>parse_dates</code> turns a text column into real dates.	Session 3
--	---	-----------

<code>pd.DataFrame({name: values, ...})</code>	Build a table from a dictionary of columns.	Session 3
<code>frame.head(n)</code> · <code>frame.tail(n)</code>	The first or last n rows, 5 by default.	Session 3
<code>frame.shape</code>	Rows and columns.	Session 3
<code>frame.dtypes</code>	What each column holds. A price column that arrived as text is a silent bug.	Session 3
<code>frame.columns</code>	The column names. <code>list(frame.columns)</code> for a plain list.	Session 3
<code>frame["close"]</code>	One column, as a Series.	Session 3
<code>frame[["date", "close"]]</code>	Several columns, as a smaller table. Note the two sets of brackets.	Session 3
<code>frame[frame["ticker"] == "AAPL"]</code>	Keep only the rows where the condition is True.	Session 3
<code>(mask_a) & (mask_b)</code> · <code>(mask_a) (mask_b)</code>	And, or. Each condition needs its own brackets.	Session 3
<code>~mask</code>	Flip a mask: every True becomes False. How you get the other half without writing the condition twice.	Session 4
<code>frame.copy()</code>	Take your own copy before modifying a selection, so pandas knows what you meant.	Session 3
<code>frame["new"] = values</code>	Assign to a name that does not exist yet, and the column appears.	Session 3
<code>s.pct_change()</code>	The percentage change from each row to the next. The first is NaN.	Session 3
<code>frame.sort_values("col")</code>	Sort the rows by a column.	Session 3
<code>frame.loc["2024-01-02":"2024-01-31"]</code>	Rows by label. With a date index you can slice with dates, and both ends are included.	Session 3
<code>frame.iloc[0]</code> · <code>frame.iloc[-1]</code>	Rows by position: the first row, the last row.	Session 3
<code>frame.iloc[[-1]]</code>	The last row, kept as a one-row table by the second pair of brackets, so a model's predict accepts it.	Session 12
<code>frame.values</code> · <code>frame.to_numpy()</code>	The plain numbers underneath, as a NumPy array, labels dropped.	Session 3
<code>frame.set_index("date")</code>	Move a column into the index, so <code>.loc</code> can slice by it.	Session 4

<code>frame.loc["2023-11-14 13:00":"2023-11-14 16:00", ["a", "b"]]</code>	Rows by date and hour on an hourly index, and columns by name, in one pair of brackets. Both ends of the slice are included.	Session 10
<code>frame.sample(n=len(frame), replace=True, random_state=0)</code>	A bootstrap sample: as many rows as the table, drawn with replacement, so some rows come twice and about a third not at all.	Session 11
<code>half = frame.sample(frac=0.5, random_state=0) · frame.drop(half.index)</code>	A random half of the rows, drawn without replacement, and the other half: the table without the rows whose labels are given.	Session 11
<code>frame.dropna()</code>	Drop every row with a missing value in any column, such as the first rows a shift leaves empty. Compare the number of rows before and after.	Session 10
<code>frame.dropna(subset=columns)</code>	Drop a row only where one of these columns is missing. Rows missing nothing but the target, such as the latest days, stay in.	Session 12
<code>frame.join(other)</code>	Add the columns of another table with the same index, each row matched by its label. <code>frame.join(pd.get_dummies(frame["weekday"], prefix="weekday", dtype=int))</code> adds the weekday columns in one step.	Session 13
<code>frame.idxmin(axis=1)</code>	For every row, the name of the column holding the smallest value: on a table of errors with one column per model, the best model on each row.	Session 12
<code>for col in frame:</code>	Looping over a table gives its column names, one at a time: this is how the columns from <code>get_dummies</code> are copied into another table.	Session 10
<code>pd.cut(s, [0, 0.2, 0.4, 1.0])</code>	Sort every value into one of the ranges given, and return the range it fell in. Group by the result to summarise each range.	Session 9
<code>pd.cut(s, edges, labels=["low", "mid", "high"])</code>	The same, with a name for each range instead of the range itself. This is how a number becomes a label with more than two values.	Session 9
<code>pd.cut(s, [-np.inf, -50, 50, np.inf])</code>	Ranges open at both ends, so every value lands in one: down by more than 50, less than 50 either way, up by more than 50.	Session 10

pandas: groups and shapes

```
import pandas as pd
```

On this card: frame is a DataFrame, s a Series, g a grouped table such as frame.groupby("ticker").

<code>frame.groupby("ticker").size()</code>	Split by a column, then count the rows in each group.	Session 3
<code>frame.groupby("ticker")["close"].mean()</code>	Split, compute inside each group, put the answers back together.	Session 3
<code>frame.groupby("ticker")["close"].pct_change()</code>	A return WITHIN each stock. Group first, or you take a return across two companies.	Session 3
<code>frame.groupby(buckets, observed=True)["y"].agg(["size", "mean"])</code>	Group by the ranges pd.cut made, keeping only ranges that have rows: how many rows fell in each range, and the share of ones among them. This is how a calibration table is built.	Session 9
<code>frame.groupby("hour")["price"].shift(1)</code>	The value one row earlier WITHIN each group. With one row per hour, grouped by the clock hour, that is the same hour the day before; .shift(2) reaches two days back. The first row of each group is NaN.	Session 10
<code>g.agg(["mean", "std", "min", "max"])</code>	Several summaries at once, as a table.	Session 3
<code>frame.pivot(index="date", columns="ticker", values="close")</code>	Reshape: one row per date, one column per ticker.	Session 3
<code>s.shift(1)</code>	Slide a column down by one, so each row can see the previous one. How you line the past up next to the present.	Session 3
<code>s.rolling(window).std()</code>	A statistic over a sliding window of rows. Where most financial features come from. Also .mean() and .sum().	Session 3
<code>s.groupby(s.index.year).mean()</code>	A dated Series grouped by the year in its index, one mean per year. From a Series of True and False, one share per year.	Session 8
<code>s.describe()</code>	Count, mean, standard deviation and the quartiles, in one call.	Session 3
<code>s.nlargest(n)</code>	The n largest values, largest first.	Session 3
<code>frame.nlargest(n, "col")</code> <code>frame.nsmallest(n, "col")</code>	The n rows with the largest, or smallest, values in one column, in that order.	Session 12

matplotlib

Every plot is the same three steps: make the axes, draw on them, then say what the reader is looking at.

```
import matplotlib.pyplot as plt
```

<code>fig, ax = plt.subplots(figsize=(9, 3))</code>	Make a figure and one axes to draw on. Every plot starts here.	Session 3
<code>fig, axes = plt.subplots(1, 2, figsize=(11, 3), sharey=True)</code>	Two axes side by side: draw on <code>axes[0]</code> and <code>axes[1]</code> . <code>sharey=True</code> gives both one vertical scale, so the two can be compared by eye.	Session 3
<code>ax.plot(x, y)</code>	A line: something over time.	Session 3
<code>ax.barh(names, values)</code>	Horizontal bars: comparing named things. Sort before you plot.	Session 3
<code>ax.invert_yaxis()</code>	Turn the vertical axis upside down, so <code>barh</code> draws the first name at the top instead of at the bottom.	Session 6
<code>ax.bar(names, values)</code>	Vertical bars, one per name: a count or a share for each group, side by side.	Session 5
<code>ax.hist(values, bins=40)</code>	A histogram: the shape of one variable.	Session 3
<code>ax.scatter(x, y)</code>	A scatter: one thing against another. <code>s=6</code> makes the dots small.	Session 3
<code>ax.plot(x, y, marker="o")</code>	A dot on every point as well as the line, for a curve with only a few points.	Session 4
<code>ax.annotate(text, (x, y), textcoords="offset points", xytext=(5, 4))</code>	Write text next to a point, 5 points to the right of it and 4 above, so the text does not cover the marker.	Session 12
<code>ax.set_title(text, loc="left")</code>	Say what the reader is looking at.	Session 3
<code>ax.set_ylabel(text)</code> · <code>ax.set_xlabel(text)</code>	Label the axes, with units. "price (USD)", not "price".	Session 3
<code>ax.plot(x, y, label="AAPL")</code> · <code>ax.legend()</code>	Name each line, then show the key. Needed as soon as there are two.	Session 3
<code>ax.set_ylim(a, b)</code>	Fix the vertical range. For bars, always start at zero.	Session 3
<code>ax.axhline(y)</code>	A horizontal reference line: zero on an error plot, or an average across bars. <code>ax.axvline(x)</code> is the vertical one.	Session 4

<code>ax.plot(x, y, drawstyle="steps-post")</code>	A staircase: each value held flat until the next one. How a forecast that takes only a few values looks over time.	Session 11
<code>ax.set_xscale("log")</code>	A logarithmic axis, so factors of ten are evenly spaced. For a validation curve over alpha or C.	Session 6
<code>fig.tight_layout()</code>	Shrink the gaps so the titles and labels of several axes do not overlap. The line before <code>plt.show()</code> .	Session 3
<code>plt.show()</code>	Display the figure. The last line of a plotting cell.	Session 3
<code>plt.figure(figsize=(11, 3))</code>	A figure with no axes of your own. <code>plot_tree</code> draws on the current figure, so make one of the right size first.	Session 11
<code>fig.savefig("name.png", dpi=200)</code>	Save it to a file. Use .pdf for something that stays sharp at any size.	Session 3

Features and a target

One row per observation, one column per feature, and one column you are trying to predict. Every model assumes it.

On this card: `s` is a Series, `X` the feature table.

<code>s.shift(-1)</code>	Move a column UP, so tomorrow's value sits on today's row. This is how a target is built.	Session 4
<code>s.rolling(20).std()</code>	A feature over the last twenty rows. It only ever looks backwards, which is what makes it legal.	Session 4
<code>(s > 0).astype(int)</code>	Turn a number into a 0/1 label, which turns a regression into a classification.	Session 4
<code>s.corr(other)</code>	How strongly two columns move together, between -1 and +1. A feature that correlates almost perfectly with the target is a leak, not a discovery.	Session 4
<code>s.diff()</code>	The difference between each row and the one before it. On logged prices this is the log return.	Session 4
<code>frame["x"] - frame.groupby("hour")["x"].shift(1)</code>	A change as a feature: this hour's value minus the same hour the day before, such as tomorrow's wind forecast minus today's. Often more telling than the level itself.	Session 10

<code>frame["change"].abs()</code>	The size of a change, sign dropped. A logistic regression cannot build this shape from the change itself, so a big move either way needs the column.	Session 10
<code>n, p = X.shape</code>	The two numbers that describe a learning problem: observations, and feature columns. The target is not a feature.	Session 4

Missing values

Real tables have holes in them. What you do about the holes changes your answer, so it is a decision rather than a formality.

<pre>import pandas as pd</pre>		
<code>pd.date_range(start, end, freq="D")</code>	Every calendar day between two dates.	Session 4
<code>frame.reindex(index)</code>	Force a table onto different row labels. Rows that did not exist arrive empty.	Session 4
<code>frame.isna().sum()</code>	Count the missing values in each column.	Session 4
<code>s.ffill()</code>	Carry the last known value forward. It only looks backwards, so a forecast may use it.	Session 4
<code>s.fillna(value)</code>	Fill with something you choose. Filling with the column mean shrinks the variance, so be deliberate.	Session 4
<code>s.interpolate()</code>	Draw a straight line across the gap. It uses the value AFTER the gap, so never in a forecast.	Session 4
<code>s.isna().astype(int)</code>	Record that a value was missing. Sometimes that fact is itself the signal.	Session 4

Extremes, and changing the scale

<pre>import numpy as np import pandas as pd</pre>		
<code>s.quantile([0.25, 0.75])</code>	The percentiles. Q1 and Q3 are the edges of the middle half of the data.	Session 4
<code>(s < q1 - 1.5 * iqr) (s > q3 + 1.5 * iqr)</code>	The IQR rule, with $iqr = q3 - q1$: flag anything outside $Q1 - 1.5 \text{ IQR}$ to $Q3 + 1.5 \text{ IQR}$. On returns it flags far more than it should.	Session 4

<code>np.log(values)</code>	The natural log. Log returns add up over time, and a log tames a long right tail.	Session 4
<code>np.exp(model.predict(X))</code>	A model fitted to <code>np.log(price)</code> forecasts the log of the price; <code>np.exp</code> turns the forecast back into kroner. <code>np.exp</code> of the difference between two such forecasts is the factor between them: 1.147 is 14.7 percent more.	Session 7
<code>(s - s.mean()) / s.std()</code>	Standardise: mean 0, standard deviation 1. Compute those two numbers on the training rows only.	Session 4
<code>pd.get_dummies(frame, columns=["sector"], dtype=int)</code>	One 0/1 column per category. This is how text gets into a feature table.	Session 4
<code>pd.get_dummies(s, prefix="weekday", dtype=int)</code>	One 0/1 column per value of a single column, named <code>weekday_0</code> , <code>weekday_1</code> and so on. A logistic regression then gives each weekday its own level, where the weekday as one number could only give it one slope across the week.	Session 10

Scoring a prediction of a number

Choose the metric before you see the answer, and always report it beside a baseline.

```
import numpy as np
```

On this card: `y` is what happened and `p` the prediction, two arrays of the same length.

<code>np.abs(y - p).mean()</code>	MAE, mean absolute error: the average size of a mistake, in the units of the target.	Session 4
<code>((y - p) ** 2).mean()</code>	MSE, mean squared error: one large miss dominates it, which is sometimes exactly what you want.	Session 4
<code>np.sqrt(((y - p) ** 2).mean())</code>	RMSE, the root of the MSE, back in the units of the target. The one to report.	Session 4
<code>1 - ((y - p) ** 2).sum() / ((y - y.mean()) ** 2).sum()</code>	R squared: how much better than always predicting the average. Zero means no better at all, and out of sample it can be negative.	Session 4
<code>np.full(len(y), y_train.mean())</code>	The baseline prediction: one number, the training average, repeated. Nothing counts as a result until it beats this.	Session 4

Scoring a prediction of a label

No residuals here. Four counts, and every classification metric is built out of them.

On this card: said and was are columns of True and False, the prediction and what happened; tp, fp, fn and tn are the four counts.

<code>(said & was).sum()</code>	True positives. False positives are (said & ~was), misses (~said & was), true negatives (~said & ~was).	Session 4
<code>(tp + tn) / (tp + fp + fn + tn)</code>	Accuracy: the share called correctly. Meaningless when one class is rare.	Session 4
<code>tp / (tp + fp)</code>	Precision: of the times the model said yes, how often was it right? Low precision means false alarms.	Session 4
<code>tp / (tp + fn)</code>	Recall: of the times it really happened, how many were caught? Low recall means misses.	Session 4
<code>2 * precision * recall / (precision + recall)</code>	F1: the harmonic mean of the two, which sits close to the smaller one.	Session 4
<code>cost_miss * fn + cost_false * fp</code>	What a set of decisions costs when the two mistakes are not worth the same. Sweep the threshold and keep the cheapest, on training rows only.	Session 9
<code>cost_false / (cost_false + cost_miss)</code>	The cheapest threshold, written down rather than swept for. It only works on probabilities that are calibrated.	Session 9

Training rows and test rows

A score computed on the rows a model learned from is not evidence.

```
import numpy as np
from sklearn.model_selection import train_test_split
```

<code>frame.loc["2022-12-31"]</code>	Split by date, never at random: the rows up to a date for training, frame.loc["2023-01-01":] for testing. A shuffled time series lets a model train on the future.	Session 4
--------------------------------------	--	-----------

<code>train_test_split(frame, test_size=0.3, random_state=0, stratify=y)</code>	Shuffle the rows and cut them in two. Right for a cross-section where rows are separate cases, wrong for a time series. stratify keeps a label's share equal in both halves, which matters when it is rare.	Session 9
<code>np.polyfit(x, y, degree)</code>	Fit a polynomial of that degree. Raising the degree always lowers training error.	Session 4
<code>np.polyval(coef, x)</code>	Evaluate a fitted polynomial, so you can score it on rows it never saw.	Session 4

Fitting a model

Four lines: import, create, fit, predict. Every model in scikit-learn takes them, so the only thing that changes later is the first one.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

On this card: X is the feature table, always with two brackets, y the target column, model a fitted model, folds a TimeSeriesSplit.

<code>frame[["col"]]</code>	A LIST of column names, so the result stays a table. X always has to be two-dimensional.	Session 5
<code>model = LinearRegression()</code>	Create the model. It installs as scikit-learn and imports as sklearn.	Session 5
<code>model.fit(X, y)</code>	Read the training rows and compute the coefficients. Features first, target second.	Session 5
<code>model.predict(X)</code>	One prediction per row of X, in the same order, as a plain NumPy array.	Session 5
<code>model.intercept_ · model.coef_</code>	What the fit found. A trailing underscore means it came from the data rather than from you.	Session 5
<code>mean_squared_error(y, p)</code>	The truth goes first. Take np.sqrt of it to get back into the units of the target.	Session 5

Choosing between models

Rows used to compare candidates can no longer give the winner an honest score. That is what the folds are for.

```
from sklearn.model_selection import TimeSeriesSplit, KFold, cross_val_score
```

On this card: X is the feature table, always with two brackets, y the target column, model a fitted model, folds a TimeSeriesSplit.

<code>folds = TimeSeriesSplit(n_splits=5)</code>	Folds that move forward in time, so every scored block comes after the rows fitted on.	Session 5
<code>TimeSeriesSplit(n_splits=5, max_train_size=500)</code>	A rolling window instead of an expanding one: the oldest rows drop out.	Session 5
<code>TimeSeriesSplit(n_splits=5, gap=20)</code>	Drop the rows whose target reaches into the block about to be scored.	Session 5
<code>KFold(n_splits=5, shuffle=True, random_state=0)</code>	The textbook folds. They assume the rows are interchangeable, which a time series is not.	Session 5
<code>for fit_rows, score_rows in folds.split(X):</code>	The row positions in each fold, as pairs. <code>cross_val_score</code> loops over this for you.	Session 5
<code>cross_val_score(model, X, y, cv=folds, scoring="neg_root_mean_squared_error")</code>	Fit and score once per fold. Errors come back negative, because every score is reported so that larger is better; put a minus in front.	Session 5

Penalised regression

Linear regression with a charge on the size of the coefficients, the scaling that charge needs, and a search over how strong it should be.

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
```

On this card: X is the feature table, always with two brackets, y the target column, model a fitted model, folds a TimeSeriesSplit. scaler is a fitted StandardScaler, pipe a fitted Pipeline, search a fitted GridSearchCV.

<code>Ridge(alpha=1000)</code>	Linear regression plus alpha times the sum of squared slopes. <code>alpha=0</code> is OLS; larger alpha pulls every slope towards zero.	Session 6
<code>Lasso(alpha=0.1)</code>	The same with a charge on the sum of absolute slopes. Sets some coefficients to exactly zero.	Session 6

<code>ElasticNet(alpha=0.1, l1_ratio=0.5)</code>	Both penalties at once. <code>l1_ratio=1</code> is the lasso, <code>l1_ratio=0</code> is ridge.	Session 6
<code>Lasso(alpha=0.001, max_iter=10000)</code>	More passes for the solver. Raise it when the <code>ConvergenceWarning</code> appears; standardise first, which is the usual cause.	Session 6
<code>scaler = StandardScaler().fit(X_train)</code>	Learn each column's mean and standard deviation from the TRAINING rows. The penalty charges by coefficient size, and size depends on units.	Session 6
<code>scaler.transform(X)</code>	Subtract the learned mean and divide by the learned standard deviation. Returns a NumPy array; use it on the test rows too, with the training numbers.	Session 6
<code>scaler.mean_ · scaler.scale_</code>	The numbers the scaler learned, one per column, in column order.	Session 6
<code>Pipeline([("scale", StandardScaler()), ("ridge", Ridge(alpha=1000))])</code>	Two steps as one model: <code>.fit</code> scales then fits, <code>.predict</code> scales with the fitted scaler then predicts. Cross-validation refits the scaler inside every fold.	Session 6
<code>pipe.named_steps["ridge"].coef_</code>	Reach the fitted object inside a step. The pipeline itself has no <code>coef_</code> .	Session 6
<code>grid = {"ridge__alpha": [1, 10, 100, 1000, 10000]}</code>	The setting to search and the values to try: step name, two underscores, argument name. Several keys multiply.	Session 6
<code>search = GridSearchCV(pipe, grid, cv=folds, scoring="neg_root_mean_squared_error")</code>	Fit every value on every fold, keep the best mean score, and refit it on all the training rows. Always pass <code>cv</code> ; the default ignores time order.	Session 6
<code>search.fit(X, y) · search.best_params_ · search.best_score_</code>	Run the search; then the winning setting, and its mean score over the folds.	Session 6
<code>search.best_estimator_ · search.predict(X)</code>	The winning pipeline, refitted on every training row, and predictions from it. Open the test rows once, here.	Session 6
<code>pd.DataFrame(search.cv_results_)</code>	The whole grid as a table: every value tried, its mean score over the folds, the spread, and its rank.	Session 6
<code>model.get_params()</code>	Every argument of an object and its current value. <code>help(Ridge)</code> prints the documentation.	Session 6

Classification

A label as the target, logistic regression, the probabilities it gives, and the scores that judge a classifier.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import cross_val_score, TimeSeriesSplit, GridSearchCV
from sklearn.metrics import (accuracy_score, confusion_matrix, precision_score,
                             recall_score, roc_curve, roc_auc_score, log_loss, f1_score,
                             average_precision_score, precision_recall_curve, brier_score_loss,
                             classification_report)
```

On this card: X is the feature table, y the label column of ones and zeros, model a fitted classifier, p the probability column model.predict_proba(X)[:, 1], predicted the 0/1 predictions.

<code>(a > b).astype(int)</code>	A label from a comparison: True and False become 1 and 0. Its mean is the share of ones, and predicting the majority class everywhere is the accuracy a model has to beat.	Session 8
<code>LogisticRegression(C=1.0, penalty="l2", solver="lbfgs", max_iter=100)</code>	The classifier, shown with its defaults: a ridge penalty of strength 1/C, solved in at most 100 steps. Create it, then .fit(X, y), as for every model.	Session 8
<code>model.intercept_ · model.coef_</code>	The fitted numbers, on the log-odds scale. coef_ has one row per boundary between classes, so coef_[0, 0] is the first slope.	Session 8
<code>-model.intercept_[0] / model.coef_[0, 0]</code>	With one column: the value of the column at which the probability is one half, where .predict() switches from 1 to 0.	Session 8
<code>np.exp(model.coef_)</code>	The odds factor: one unit more of a column multiplies the odds of a 1 by this. A coefficient adds to the log-odds; its exponential multiplies the odds.	Session 8
<code>model.classes_</code>	The classes, in the order predict_proba uses for its columns.	Session 8
<code>model.predict_proba(X)</code>	One row per observation, one column per class, each row summing to one.[:, 1] is the probability of a 1.	Session 8
<code>model.predict(X)</code>	The 0/1 predictions: 1 where the probability of a 1 is at least one half.	Session 8

<code>(p >= threshold).astype(int)</code>	Predictions at any other threshold, from the probability column. The model is unchanged; only the comparison changed.	Session 8
<code>model.n_iter_</code>	How many steps the solver took. Log-loss has no formula for the coefficients, so <code>.fit()</code> improves them step by step; raise <code>max_iter</code> when the <code>ConvergenceWarning</code> appears, or standardise first.	Session 8
<code>log_loss(y, p)</code>	The objective, averaged over the rows: minus the log of the probability given to what happened. A confident wrong probability costs the most.	Session 8
<code>accuracy_score(y, predicted)</code>	The share of predictions that match the label. True labels first, predictions second, like every metric. Compare it with the majority rule.	Session 8
<code>confusion_matrix(y, predicted)</code>	A 2 by 2 array: rows are what happened, columns are the prediction, both in the order of classes_. Top left true negatives, bottom right true positives.	Session 8
<code>precision_score(y, predicted)</code>	The share of predicted 1s that were 1. Falls when the threshold falls.	Session 8
<code>recall_score(y, predicted)</code>	The share of real 1s that were predicted. Falls when the threshold rises.	Session 8
<code>roc_curve(y, p)</code>	Returns fpr, tpr, thresholds: the false positive rate and the recall at every threshold where a point moves. Joined up, the ROC curve.	Session 8
<code>roc_auc_score(y, p)</code>	The area under the ROC curve: 0.5 for a random ranking, 1 for a perfect one, no threshold involved. Pass the probabilities, never the 0/1 predictions.	Session 8
<code>cross_val_score(model, X, y, cv=folds, scoring="roc_auc")</code>	One score per fold; larger is better, so no minus sign. Other strings: "accuracy", "neg_log_loss", "precision", "recall".	Session 8
<code>LogisticRegression(C=0.01)</code>	The strength of the penalty, upside down: $C = 1/\alpha$, so a small C is a strong penalty. Standardise in a pipeline first and choose C on a grid in factors of ten.	Session 8
<code>{"logit__C": [0.0001, 0.001, 0.01, 0.1, 1, 10]}</code>	A grid for <code>GridSearchCV</code> : the step name, two underscores, the argument. Then <code>GridSearchCV(pipe, grid, cv=folds, scoring="roc_auc").fit(X, y)</code> , and <code>best_params_</code> , <code>best_score_</code> , <code>predict_proba</code> as before.	Session 8

<code>LogisticRegression(penalty="l1", solver="liblinear", C=0.01)</code>	The lasso's penalty on a classifier, which sets coefficients to exactly zero. It needs a solver that can handle absolute values; the default lbfgs cannot. scikit-learn 1.8 and later retire penalty: there it is <code>LogisticRegression(l1_ratio=1, solver="liblinear", C=0.01)</code> .	Session 8
<code>confusion_matrix(y, predicted).ravel()</code>	The 2 by 2 array flattened into tn, fp, fn, tp, in reading order. The four counts every other score is built from.	Session 9
<code>LogisticRegression(class_weight="balanced")</code>	Count each class in inverse proportion to its size while fitting. Raises recall on a rare class and lowers accuracy, leaves the AUC alone, and breaks the probabilities.	Session 9
<code>average_precision_score(y, p)</code>	The whole precision-recall curve in one number. Its no-information line is the share of ones, not 0.5, so report the two together. More informative than AUC when the class is rare.	Session 9
<code>precision_recall_curve(y, p)</code>	Returns precision, recall and the thresholds, for drawing the curve or reading the precision at a recall you care about.	Session 9
<code>brier_score_loss(y, p)</code>	The mean squared error of the probabilities, with what happened as 1 or 0. Smaller is better. It separates two models that the AUC cannot tell apart.	Session 9
<code>CalibratedClassifierCV(model, method="sigmoid", cv=5)</code>	Wrap a model and learn a second, small model mapping its numbers onto probabilities, on folds. It repairs a distorted level; it cannot repair a base rate that has changed since fitting.	Session 9
<code>classification_report(y, predicted, labels=order, digits=3)</code>	Precision, recall and F1 for every class, with the macro and weighted averages underneath. labels= sets the order.	Session 9
<code>f1_score(y, predicted, average="macro")</code>	One F1 per class, averaged equally. "weighted" averages by class size instead, and None returns the scores themselves. Read macro when the class you care about is the small one.	Session 9
<code>confusion_matrix(y, predicted, labels=order)</code>	The matrix with the rows and columns in an order you choose rather than alphabetically. Without it the middle row of a three-class matrix is rarely the class you expect.	Session 9

<code>model.decision_function(X)</code>	The raw score per class, before the exponential and the division that turn scores into probabilities.	Session 9
<code>np.exp(scores) / np.exp(scores).sum()</code>	The softmax: make every score positive, then divide by the total so they add to one. With two classes it is the sigmoid.	Session 9
<code>move[p >= t].sum() - move[p <= 1 - t].sum() - cost * n_trades</code>	A bet on the direction, in money: long where the probability of a rise is at least t, short where it is at most 1 - t, every trade paying a cost. The best t rises with the cost; choose it on the training rows.	Session 10

k-nearest neighbours

A classifier with no coefficients: it stores the training rows and lets the nearest ones vote.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
```

On this card: X is the feature table, always with two brackets, y the target column, model a fitted model, folds a TimeSeriesSplit.

<code>KNeighborsClassifier(n_neighbors=k)</code>	Classify a row by a vote among the k training rows closest to it. Works for any number of classes with no change.	Session 9
<code>Pipeline([("scale", StandardScaler()), ("knn", KNeighborsClassifier())])</code>	The scaler is not optional here: distance is dominated by whichever column has the largest spread, so an unscaled model reads only that one.	Session 9
<code>k, as the dial</code>	A small k fits the training rows and little else; k of 1 scores a perfect training AUC. A large k is a simple model, the way a small C was.	Session 9
<code>{"knn__n_neighbors": [1, 5, 15, 51, 151, 301]}</code>	A grid for GridSearchCV. k cannot exceed the SMALLEST fold, not the training size; above that the search returns nan and warns.	Session 9
<code>model.predict_proba(X)</code>	The share of the k neighbours in each class, so the probabilities are multiples of 1/k. A small k gives coarse ones.	Session 9
<code>["wind_change", "weekday", "hour"]</code>	k-NN can take a category as one plain number: once the columns are scaled, the nearest rows to a Monday at 18:00 are Mondays near 18:00. A logistic regression needs the weekday as columns instead.	Session 10

Trees and forests

Models that ask questions of one column at a time instead of fitting slopes: one tree, many trees averaged, and the settings that decide how big they grow.

```
import time
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier, plot_tree
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor, RandomForestClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
```

On this card: X is the feature table, y the target, columns the list of feature names, tree a fitted tree, forest a fitted forest, model the model a search tunes, grid a dictionary of settings to try, folds a TimeSeriesSplit.

<code>DecisionTreeRegressor(max_depth=2, random_state=0)</code>	A tree for a number. Each question compares one column with a cut, and each leaf forecasts the mean of its training rows. max_depth caps the questions on any path; without it the tree splits until it fits the training rows almost exactly.	Session 11
<code>DecisionTreeRegressor(min_samples_leaf=100)</code>	Another way to keep a tree small: no leaf may hold fewer training rows than this. Choose it, or max_depth, on the folds.	Session 11
<code>DecisionTreeRegressor(ccp_alpha=0.02)</code>	Grow the tree, then prune it: a question stays only if it lowers the training error by at least ccp_alpha for each leaf it adds. The default, 0, keeps every question.	Session 11
<code>DecisionTreeClassifier(max_depth=2, random_state=0)</code>	A tree for a label. The same cuts, chosen by Gini, $2p(1 - p)$ for two classes, which is twice the squared error of the 0s and 1s per row. Each leaf holds the share of each class, which predict_proba returns.	Session 11
<code>plot_tree(tree, feature_names=columns, filled=True, impurity=False, precision=3, fontsize=9)</code>	Draw a fitted tree with matplotlib: every box shows its question, the rows that reached it and its forecast. filled shades the boxes by their forecast, impurity=False drops the error line, and label="root" names the fields in the top box only.	Session 11
<code>plot_tree(tree, class_names=["repaid", "defaulted"], proportion=True)</code>	For a classification tree: name the classes in the order of classes_, and show each box's shares instead of its counts.	Session 11
<code>tree.get_depth()</code> · <code>tree.get_n_leaves()</code>	The questions on the longest path, and the number of leaves. A tree grown without a limit ends with about one leaf per training row.	Session 11

<code>tree.tree_.feature[0]</code> <code>tree.tree_.threshold[0]</code>	Inside a fitted tree: the column number and the cut of the first question, so <code>columns[tree.tree_.feature[0]]</code> is its name. The boxes are numbered depth first, and a leaf has feature -2.	Session 11
<code>tree.apply(X)</code>	The number of the leaf each row lands in. Group any column by it to see what each leaf holds.	Session 11
<code>BaggingRegressor(DecisionTreeRegressor(), n_estimators=25, random_state=0)</code>	Bagging: grow a deep tree on each of 25 bootstrap samples of the training rows and average their forecasts, which cancels much of what one tree gets wrong by chance.	Session 11
<code>forest.estimators_</code>	The fitted trees inside a bagging model or a forest, as a list, each a tree like any other. They were fitted on plain arrays, so give their <code>predict X.to_numpy()</code> .	Session 11
<code>RandomForestRegressor(n_estimators=100, max_features="sqrt", min_samples_leaf=50, random_state=0)</code>	A forest: bagging in which each question chooses among a few columns drawn at random, here the square root of their number, so the trees differ. The default for a number, <code>max_features=1.0</code> , is plain bagging; <code>RandomForestClassifier</code> 's default is already "sqrt".	Session 11
<code>RandomForestRegressor(oob_score=True)</code> <code>forest.oob_prediction_</code>	Out-of-bag: each training row is forecast by the trees whose sample left it out, so the forest scores itself without folds. Fair on rows that are separate cases; too kind on days, whose neighbours were in the sample.	Session 11
<code>RandomForestClassifier(n_estimators=100, min_samples_leaf=50, oob_score=True)</code>	The forest for a label. <code>forest.oob_decision_function[:, 1]</code> is each training row's out-of-bag probability of a 1.	Session 11
<code>RandomForestRegressor(n_jobs=-1)</code>	Grow the trees on every core at once. <code>GridSearchCV</code> takes <code>n_jobs</code> too, and then fits the folds side by side instead.	Session 11
<code>{"max_depth": [1, 2, 3, 4, 6, 8]}</code>	A grid for a model on its own, outside a pipeline: the argument name with no step name in front. <code>GridSearchCV(DecisionTreeRegressor(random_state=0), grid, cv=folds, ...)</code> as before.	Session 11
<code>RandomizedSearchCV(model, grid, n_iter=10, cv=folds, scoring="neg_root_mean_squared_error", random_state=0)</code>	Try <code>n_iter</code> combinations drawn at random from the grid instead of every one; <code>best_params_</code> and <code>best_score_</code> as for <code>GridSearchCV</code> . For grids too big to run in full.	Session 11

```
len(grid["a"]) * len(grid["b"]) * 5 + 1
```

The fits a search will make: one per combination on each of five folds, and one refit on all the training rows. Time one fit and multiply before you press run.

Session 11

```
start = time.perf_counter()
```

Read a clock, in seconds. `time.perf_counter() - start` after a fit is how long it took.

Session 11

Boosted trees

Small trees added one at a time, each fitted to what the trees before it still get wrong, the libraries that do this fast, and two ways to read which columns a model uses.

```
from sklearn.ensemble import (GradientBoostingRegressor, GradientBoostingClassifier,
                              HistGradientBoostingRegressor, HistGradientBoostingClassifier)
from sklearn.inspection import permutation_importance
from xgboost import XGBRegressor, XGBClassifier
from lightgbm import LGBMRegressor, LGBMClassifier, early_stopping
```

On this card: `X` and `y` are the rows a model is fitted on, `X_stop` and `y_stop` later rows held back to stop it, `X_test` and `y_test` the test rows, columns the list of feature names, `boost` a fitted boosting model, `result` what `permutation_importance` returns.

```
GradientBoostingRegressor(n_estimators=100,
learning_rate=0.1, max_depth=1,
random_state=0)
```

Boosting: start from the mean of `y`, then add `n_estimators` small trees one at a time, each fitted to the residuals of the forecast so far and shrunk by `learning_rate`. `max_depth=1` makes every tree a stump; the default is 3. A smaller `learning_rate` needs more trees for the same fit.

Session 12

```
GradientBoostingClassifier(n_estimators=100
, learning_rate=0.1, max_depth=3)
```

The same for a label. The trees are added on the log-odds scale, each fitted to `y` minus the probability so far, and `predict_proba` turns the sum into probabilities.

Session 12

```
List(boost.staged_predict(X))
```

The forecast after 1, 2, 3, ... trees, up to `n_estimators`. `staged_predict` hands them out one at a time, so `list()` collects them: `[49]` is the forecast after 50 trees.

Session 12

```
GradientBoostingRegressor(n_estimators=5000
, validation_fraction=0.1,
n_iter_no_change=10)
```

Early stopping inside scikit-learn: hold back a random tenth of the training rows and stop once 10 more trees have not helped on them. `boost.n_estimators_` is the number of trees it grew. A random tenth suits separate cases and is too kind on days.

Session 12

<code>HistGradientBoostingRegressor(max_iter=200, max_depth=2)</code> · <code>HistGradientBoostingClassifier()</code>	scikit-learn's fast boosting: each column is first sorted into at most 255 bins, and cuts fall only between bins. <code>max_iter</code> is the number of trees. Above 10,000 rows it stops early on its own, on a random tenth of them, unless <code>early_stopping=False</code> . It has no <code>feature_importances_</code> .	Session 12
<code>XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=1, random_state=0)</code>	XGBoost, with the methods every scikit-learn model has: <code>fit</code> , <code>predict</code> , and <code>predict_proba</code> on <code>XGBClassifier</code> . It cuts each column only between 256 bins and puts a light ridge penalty on the leaf values. Its own defaults are <code>learning_rate=0.3</code> and <code>max_depth=6</code> , so set both.	Session 12
<code>XGBRegressor(n_estimators=3000, learning_rate=0.1, early_stopping_rounds=50)</code>	Early stopping in XGBoost: score every new tree on the rows given to fit as <code>eval_set</code> , and stop 50 trees after the best score. <code>n_estimators</code> is then only a ceiling.	Session 12
<code>boost.fit(X, y, eval_set=[(X_stop, y_stop)], verbose=False)</code>	The rows that stop it, as a list of (columns, target) pairs: rows later than those fitted on, never the test rows. <code>verbose=False</code> keeps it from printing the score of every tree.	Session 12
<code>boost.best_iteration</code> · <code>boost.best_score</code>	Where the score on <code>eval_set</code> was best, counting the first tree as 0, and that score. <code>predict</code> then uses the first <code>best_iteration + 1</code> trees.	Session 12
<code>XGBRegressor(n_estimators=boost.best_iteration + 1, learning_rate=0.1, max_depth=3)</code>	The number of trees early stopping found, refitted on all the training rows, the block that chose it included. The refit takes no <code>eval_set</code> and no <code>early_stopping_rounds</code> .	Session 13
<code>boost.eval_result()["validation_0"]</code> <code>["rmse"]</code>	XGBoost's record of the score after every tree, for each <code>eval_set</code> entry, under <code>validation_0</code> , <code>validation_1</code> , ... in the order given. Plot it against the number of trees to see the curve early stopping read.	Session 12
<code>XGBClassifier(eval_metric="auc", early_stopping_rounds=50)</code>	The score early stopping watches. The default is the log loss for a label and the RMSE for a number; "auc" stops at the best AUC instead.	Session 12
<code>XGBRegressor(reg_lambda=1, subsample=0.8, colsample_bytree=0.8)</code>	<code>reg_lambda</code> is the ridge penalty on the leaf values. <code>subsample</code> and <code>colsample_bytree</code> give each tree a random share of the rows and of the columns, as a forest does. Settings to try after the depth and the number of trees, not before.	Session 12

<code>boost.predict(X, output_margin=True)</code>	XGBClassifier: the sum of the trees before the sigmoid, on the log-odds scale. $1 / (1 + \text{np.exp}(-\text{margin}))$ gives back <code>predict_proba(X)[:, 1]</code> .	Session 12
<code>XGBRegressor(monotone_constraints={"area": 1})</code>	Never let the forecast fall as a column rises, everything else unchanged: 1 for a column that may only raise the forecast, -1 for one that may only lower it, by its name in the table given to fit.	Session 13
<code>LGBMRegressor(n_estimators=100, learning_rate=0.1, num_leaves=2, verbose=-1)</code>	LightGBM. It grows each tree leaf by leaf, splitting whichever leaf lowers the error most, so <code>num_leaves</code> sets the size of a tree: 2 makes a stump, and the default is 31. <code>verbose=-1</code> silences its messages.	Session 12
<code>boost.fit(X, y, eval_set=[(X_stop, y_stop)], eval_metric="auc", callbacks=[early_stopping(50, verbose=False)])</code>	Early stopping in LightGBM: a callback passed to fit rather than a setting of the model. <code>boost.best_iteration_</code> counts the first tree as 1, where XGBoost's <code>best_iteration</code> counts it as 0.	Session 12
<code>boost.best_score_["valid_0"]["l2"] ** 0.5</code>	LightGBM's best score, by data set and metric. For a number the metric is the squared error, l2, so its square root is the RMSE.	Session 12
<code>pd.Series(boost.feature_importances_, index=columns)</code>	How much the model used each column, one number per column in column order. scikit-learn's trees sum each column's fall in the error and scale the total to one, XGBoost averages the fall per cut, and LightGBM counts the cuts. Measured on the training rows, so a column of noise can still collect many cuts.	Session 12
<code>LGBMClassifier(importance_type="gain")</code>	Make LightGBM sum the fall in the loss on each column instead of counting its cuts. XGBoost takes <code>importance_type</code> too, where "weight" counts the cuts.	Session 12
<code>permutation_importance(boost, X_test, y_test, scoring="roc_auc", n_repeats=10, random_state=0)</code>	For any model: shuffle one column at a time on rows the model did not fit, and record how far the score falls, <code>n_repeats</code> times for each column. A column of noise falls by about zero.	Session 12
<code>result.importances_mean</code> <code>result.importances_std</code>	Per column, in column order: the mean fall in the score over the repeats, and its spread. <code>pd.Series(result.importances_mean, index=columns)</code> names them.	Session 12

Reading error messages

The last line names the problem. Read it before you change anything: it is almost always telling you the truth.

NameError	A name Python has never seen. Usually a typo, or a cell you have not run yet.	Session 1
TypeError	The right operation on the wrong kind of thing. Often text where a number belongs, or a placeholder ... left in a cell.	Session 1
IndexError	A position that does not exist. A list of 5 stops at index 4.	Session 1
KeyError	A dictionary key, or a column name, that is not there.	Session 2
SyntaxError	Python could not read the line at all. Look for a missing bracket, quote or colon.	Session 1
ZeroDivisionError	Dividing by zero. Often an empty list you thought had something in it.	Session 1
ModuleNotFoundError	The package is not installed, or you are running a different Python.	Session 3
FileNotFoundError	The path is wrong relative to where you are running from.	Session 3
ValueError, on shapes	Two arrays whose dimensions do not fit. Check .shape on both.	Session 3
LinAlgError	A matrix with no inverse: one column carries nothing the others do not. Perfect multicollinearity.	Session 3
ValueError, on feature names	The columns given to predict are not the ones fit saw, in the same order. Select them with the same list both times.	Session 11
ValueError, on early stopping	Must have at least 1 validation dataset: XGBoost was given early_stopping_rounds but no eval_set to score the trees on. Pass eval_set to fit, or leave early_stopping_rounds out of a refit.	Session 12
ConvergenceWarning	Not an error: the solver ran out of steps before the coefficients settled. Raise max_iter, or standardise the columns first.	Session 6